

UNIVERSIDAD POLITÉCNICA DE MADRID

Programación Declarativa

Hora Punta (Rush Hour)

Memoria de la Práctica 4

Autores: Pablo Campo Herrero y Rubén Jaraba Sánchez

Matriculas: 23M056 y 23M070

Tabla de contenido

1. Introducción	3
2. Modelo de datos.....	3
3. Lectura y parseo del tablero.....	4
3.1. Validación de los datos de entrada	4
4. Lógica de movimiento	5
5. El resolvidor.....	6
6. Clasificador de dificultad	6
6.1. Número de pasos y número de piezas	6
6.2. Proporción de piezas movidas.....	7
6.3. Grado de simetría.....	7
6.4. El polinomio y su calibración	7
7. Conclusiones.....	8
8. Enlace Repositorio de GitHub del trabajo	8

1. Introducción

Rush Hour (en esta práctica, "Hora punta") es un rompecabezas de bloques deslizantes inventado por Nob Yoshigahara en la década de 1970. Simula un aparcamiento de 6x6 casillas en el que se colocan coches (que ocupan dos posiciones) y camiones (que ocupan tres). La meta es conseguir que un coche concreto, en nuestro caso el coche rojo, representado siempre con la letra "A" alcance la salida, situada en el borde derecho del tablero, moviendo el resto de vehículos. Un movimiento consiste en desplazar un único vehículo una posición hacia delante o hacia atrás según su orientación, si está colocado en horizontal se moverá de izquierda a derecha y si es vertical se moverá hacia arriba y hacia abajo, siempre que la casilla de destino esté libre y dentro del tablero.

El objetivo es desarrollar un programa que, dada una situación inicial, la resuelva y la clasifique según su dificultad en cuatro niveles: principiante, intermedio, avanzado y experto. La clasificación se basa en un polinomio basado que utiliza características de la solución mínima: número de pasos, proporción de piezas movidas, grado de simetría y número de piezas en la situación inicial, entre otras. Para contrastar la fórmula se dispone del fichero RushHour.txt, con 32 tableros ya clasificados por el profesor en las cuatro categorías.

El código se ha organizado en dos módulos: RushHour.hs, con todo el modelo de datos y la lógica del juego, y Main.hs, que carga el fichero de tableros y ejecuta el análisis completo sobre cada uno de ellos.

2. Modelo de datos

Una posición se representa como un par (fila, columna), ambos en el rango 0-5 puesto que el tablero es siempre 6x6:

```
type Posicion = (Int, Int)
```

Un vehículo puede ser Horizontal (ocupa varias columnas de una misma fila) o Vertical (ocupa varias filas de una misma columna). Cada vehículo guarda su letra identificativa, su longitud (2 para coches, 3 para camiones), su orientación y una única posición, que es la celda más a la izquierda si es horizontal o la más arriba si es vertical. El resto de celdas que ocupa el vehículo se derivan siempre de esa ancla, nunca se almacenan de forma redundante:

```
data Orientacion = Horizontal | Vertical deriving (Show, Eq)

data Vehiculo = Vehiculo {
  idVehiculo  :: Char,
  longitud    :: Int,
  orientacion :: Orientacion,
  posicion    :: Posicion
} deriving (Show, Eq)
```

El tablero completo es la lista de todos sus vehículos, usamos newtype para poder definirle instancias propias de Show y Read distintas de las de una lista normal:

```
newtype Tablero = Tablero [Vehiculo] deriving (Eq)
```

3. Lectura y parseo del tablero

Cada tablero llega como una cadena de 36 caracteres (6 filas de 6 columnas, leído por filas). La función `agruparEn` trocea esa cadena en 6 listas de 6 caracteres, y `cuadrículaDe` la convierte en una lista de pares (posición, carácter), descartando las celdas vacías 'o':

```
agruparEn :: Int -> [a] -> [[a]]
agruparEn _ [] = []
agruparEn n xs = take n xs : agruparEn n (drop n xs)

cuadrículaDe :: String -> [(Posicion, Char)]
cuadrículaDe cadena =
  [ ((f, c), caracter)
  | (f, fila)      <- zip [0..5] (agruparEn 6 cadena)
  , (c, caracter) <- zip [0..5] fila
  , caracter /= 'o'
  ]
```

A partir de esa cuadrícula, `leerCadena` identifica cada letra distinta como un vehículo y `crearVehiculo` reconstruye cada uno: busca todas las celdas donde aparece su letra, las ordena y asume que forman una línea recta contigua, deduciendo la orientación de si las dos primeras celdas comparten fila o columna.

El tipo `Tablero` se ha hecho instancia de la clase `Read`, de forma que `"read cadena :: Tablero"` toma los primeros 36 caracteres de la entrada.

3.1. Validación de los datos de entrada

Al ejecutar el solucionador sobre los 32 tableros de `RushHour.txt` se detectó que muchos de ellos no se podían resolver. Al revisar los casos de prueba a mano, comprobamos que una gran parte de los ejemplos propuestos no tenían sentido, es decir los coches estaban partidos o había vehículos que ocupaban una sola casilla y también había algún error humano tipográfico que confunde alguna letra.

Tras solucionar estos problemas, cambiando los tableros con errores claros y eliminando los que daban problemas, todavía había algunos tableros que no éramos capaces de resolver pero que no veíamos el error. Investigando la causa, se comprobó que `crearVehiculo` no verifica que las celdas encontradas para una letra formen realmente una línea recta: sencillamente asume una forma contigua a partir de las dos primeras celdas ordenadas, pero si es un camión no se comprueba dónde está colocada la tercera letra.

Para solucionarlo añadimos una función de validación que compara, para cada vehículo, sus celdas reales con las celdas que se deberían ser basándonos en la posición principal y la longitud. Si no coinciden, se informa del problema en lugar de tratar el tablero como si fuera irresoluble:

```
validarTablero :: String -> Tablero -> Bool

validarTablero cadena (Tablero vehiculos) =
  let errores = concat (map chequear vehiculos)
```

```

    in if null errores
      then True
      else error ("Tablero invalido: " ++ intercalate "; " errores)
  where
    cuadricula = cuadriculaDe cadena
    chequear v =
      let real = sort [pos | (pos, c) <- cuadricula, c == idVehiculo v] -- Celdas
que ocupa realmente el vehiculo
      reconstruido = sort (posicionesVehiculo v) -- Celdas que deberia ocupar
    in if real == reconstruido
      then []
      else ["Vehiculo " ++ [idVehiculo v] ++ " mal formado"]

```

Main.hs llama a validarTablero antes de resolver cada tablero; si detecta datos corruptos, imprime un aviso y continúa con el siguiente tablero, en vez de abortar la ejecución o confundir el fallo con un puzzle sin solución.

4. Lógica de movimiento

posicionesVehiculo calcula todas las celdas que ocupa un vehículo a partir de su posición principal: si es horizontal se extiende hacia la derecha (columna + i), si es vertical hacia abajo (fila + i). enLimites comprueba que una posición esté dentro del tablero 6x6, y estaLibre comprueba que ninguno de los vehículos dados ocupe una posición concreta.

```

posicionesVehiculo :: Vehiculo -> [Posicion]
posicionesVehiculo (Vehiculo _ long Horizontal (f, c)) = [(f, c + i) | i <- [0 ..
long - 1]]
posicionesVehiculo (Vehiculo _ long Vertical (f, c))    = [(f + i, c) | i <- [0 ..
long - 1]]

```

intentarMover desplaza un vehículo una única celda en la dirección indicada (Adelante o Atrás) y comprueba que todas las celdas resultantes estén dentro del tablero y libres de otros vehículos. Devuelve Just el vehículo movido si el movimiento es válido, o Nothing en caso contrario. A partir de ahí, siguientesTableros genera todos los tableros alcanzables en un único movimiento probando cada vehículo en sus dos direcciones posibles:

```

siguientesTableros :: Tablero -> [Tablero]
siguientesTableros (Tablero vehiculos) =
  let b = Tablero vehiculos
      [ Tablero (map (\coche -> if idVehiculo coche == idVehiculo v then vMovido
else coche) vehiculos)
      | v <- vehiculos

```

```
, dir <- [Adelante, Atras]
, Just vMovido <- [intentarMover v dir b]
]
```

5. El resolutor

estaResuelto comprueba si el coche 'A' tiene su posición principal en la columna 4: como el coche mide 2 celdas, eso significa que ocupa las columnas 4 y 5, es decir, está pegado al borde derecho del tablero (la salida).

resolver implementa una búsqueda en anchura sobre el grafo de tableros: cada tablero es un nodo y cada movimiento válido es una arista hacia otro tablero. BFS explora los estados por niveles (primero todos los alcanzables en 1 movimiento, luego en 2, etc.), lo que garantiza que la primera solución encontrada use el número mínimo de movimientos posible. La cola de la búsqueda no almacena tableros sueltos, sino caminos completos desde el tablero inicial, de forma que al llegar a la meta se dispone directamente de toda la secuencia de pasos de la solución. Un conjunto de visitados evita volver a explorar tableros ya vistos:

```
resolver :: Tablero -> Maybe [Tablero]
resolver tableroInicial = bfs [[tableroInicial]] []
  where
    bfs :: [[Tablero]] -> [Tablero] -> Maybe [Tablero]
    bfs [] _ = Nothing
    bfs ((tableroActual:restoCola):cola) visitados
      | estaResuelto tableroActual = Just (reverse (tableroActual:restoCola))
      | tableroActual `elem` visitados = bfs cola visitados
      | otherwise =
          let siguientesOpciones = siguientesTableros tableroActual
              siguientesValidos = filter (`notElem` visitados) siguientesOpciones
              nuevasRutas = [sigTablero : (tableroActual:restoCola) |
sigTablero <- siguientesValidos]
          in bfs (cola ++ nuevasRutas) (tableroActual : visitados)
```

Se ha comprobado, ejecutando el programa sobre los 27 tableros bien formados del fichero, que el solucionador encuentra siempre una solución y que el número de pasos crece de forma coherente entre niveles de dificultad (de 10 pasos en el tablero BEGINNER más sencillo hasta 65 pasos en el EXPERT más difícil).

6. Clasificador de dificultad

Para calcular la dificultad de los tableros hemos tomado en cuanto 4 factores: número de pasos, proporción de piezas movidas, grado de simetría y número de piezas en la situación inicial. Se han implementado las cuatro:

6.1. Número de pasos y número de piezas

```
pasosSolucion :: [Tablero] -> Int
pasosSolucion ruta = length ruta - 1
```

```
totalVehiculos :: Tablero -> Int
totalVehiculos (Tablero vehiculos) = length vehiculos
```

6.2. Proporción de piezas movidas

Se recorre la ruta solución comparando cada tablero con el siguiente para averiguar qué vehículo cambia de posición en cada paso (en cada movimiento se mueve exactamente uno). La proporción es el número de vehículos distintos que llegan a moverse alguna vez entre el total de vehículos del tablero:

```
vehiculosMovidos :: [Tablero] -> [Char]
vehiculosMovidos ruta = nub (concat (zipWith difieren ruta (tail ruta)))
  where
    difieren (Tablero vs1) (Tablero vs2) =
      [idVehiculo v2 | (v1, v2) <- zip vs1 vs2, posicion v1 /= posicion v2]

proporcionPiezasMovidas :: [Tablero] -> Float
proporcionPiezasMovidas ruta =
  let total = totalVehiculos (head ruta)
      movidos = length (vehiculosMovidos ruta)
  in if total == 0 then 0 else fromIntegral movidos / fromIntegral total
```

6.3. Grado de simetría

Se calcula sobre la situación inicial, no sobre la solución. Se toman todas las celdas ocupadas del tablero y, para tres simetrías naturales de una rejilla cuadrada (reflejo horizontal, reflejo vertical y rotación de 180 grados), se mide qué proporción de celdas ocupadas tiene también ocupada su celda transformada. El grado de simetría es el máximo de esas tres proporciones (0 = nada simétrico, 1 = totalmente simétrico):

```
gradoSimetria :: Tablero -> Float
gradoSimetria (Tablero vehiculos) =
  let ocupadas = concatMap posicionesVehiculo vehiculos
      total = length ocupadas
      reflejoH (f, c) = (f, 5 - c)
      reflejoV (f, c) = (5 - f, c)
      rotacion180 (f, c) = (5 - f, 5 - c)
      ratio transformar =
        fromIntegral (length (filter (\p -> transformar p `elem` ocupadas)
          ocupadas))
          / fromIntegral total
  in if total == 0 then 0 else maximum [ratio reflejoH, ratio reflejoV, ratio
    rotacion180]
```

6.4. El polinomio y su calibración

Los cuatro factores se combinan en una puntuación ponderada, y tres umbrales dividen esa puntuación en las cuatro categorías:

```
puntuacion = pasos
  + 0.6 * numVehiculos
  + 20 * propMovidas
  + 8 * simetria
```

```
if puntuacion < 68.0 then Principiante
else if puntuacion < 72.0 then Intermedio
else if puntuacion < 83.0 then Avanzado
else Experto
```

Los coeficientes y los umbrales no se han elegido a ciegas: se ha ejecutado el programa sobre los 27 tableros no corruptos de RushHour.txt, se ha comparado la clasificación obtenida con la categoría real indicada por las cabeceras del fichero (BEGINNER / INTERMEDIATE / ADVANCED / EXPERT), y se han ajustado pesos y umbrales para maximizar el acierto. El resultado es un 78% de acierto (21 de 27 tableros bien clasificados).

Durante esa calibración se observó un hallazgo relevante: la categoría EXPERT se separa con total nitidez del resto porque hay un salto claro en la puntuación entre los tableros ADVANCED más difíciles y los EXPERT más fáciles. Sin embargo, las categorías Intermedio y Avanzado se solapan sustancialmente en los datos de partida bajo cualquiera de las dos convenciones: existen tableros ADVANCED con menos pasos, menos piezas y menor simetría que algunos tableros INTERMEDIATE. Esto nos hace pensar que los ejemplos son algo representativo y que es casi imposible clasificar a la perfección la dificultad de los tableros.

7. Conclusiones

El núcleo del programa resuelve correctamente los tableros bien formados del fichero de pruebas, incluyendo los más complejos (hasta 65 movimientos y 13 vehículos). Se ha añadido una validación de los datos de entrada que detecta y reporta con claridad los tableros con datos corruptos del fichero.

La fórmula de dificultad incorpora los cuatro factores pedidos en el enunciado (pasos, proporción de piezas movidas, grado de simetría y número de piezas) y, calibrada contra los ejemplos ya clasificados, acierta el 78% de los tableros válidos. El nivel Experto se identifica sin ambigüedad; los niveles Intermedio y Avanzado son los que concentran los errores, debido a un solapamiento real en los datos de partida que ninguna combinación lineal de estas cuatro variables logra resolver del todo.

Como posibles mejoras futuras se plantean:

1. Incorporar alguna característica estructural adicional que ayude a separar Intermedio de Avanzado (por ejemplo, la profundidad del árbol de bloqueos que impiden el movimiento directo del coche rojo).
2. Mejorar el rendimiento de la búsqueda en anchura sustituyendo la lista de visitados por una estructura con búsqueda más eficiente para tableros con muchos vehículos.

8. Enlace Repositorio de GitHub del trabajo

Presentamos un enlace al repositorio donde guardamos el trabajo, para facilitar su visualización

<https://github.com/rjaraba2005/practica-4-Julio>